



PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
FORMATO GUÍA DE APRENDIZAJE – ACTIVIDADES
MODELAR LAS FUNCIONES DEL SOFTWARE

Actividad: Conversión de Modelo de Clases UML a Código en Lenguaje de programación seleccionado

Objetivo: Conocer los conceptos relacionados con el diseño y programación orientada a objetos y su respectiva aplicación.

Fundamentos de Programación Orientada a Objetos en Python

Clase y Objeto

```
class Persona:  
    pass  
  
p1 = Persona()
```

Atributos y Constructor

```
class Persona:  
    def __init__(self, nombre, edad):  
        self.nombre = nombre  
        self.edad = edad
```

Métodos

```
def saludar(self):  
    return f"Hola, soy {self.nombre}"
```



Encapsulamiento

```
class Cuenta:
    def __init__(self, saldo):
        self.__saldo = saldo # atributo privado

    def depositar(self, valor):
        self.__saldo += valor
```

Herencia

```
class Estudiante(Persona):
    def estudiar(self):
        print("Estudiando...")
```

Polimorfismo

```
class Animal:
    def hablar(self):
        pass

class Perro(Animal):
    def hablar(self):
        return "Guau"
```

Relaciones UML

- Asociación
- Agregación
- Composición
- Generalización (Herencia)



Ejercicio Guiado – Conversión Modelo UML a Código

Caso: Sistema de Biblioteca

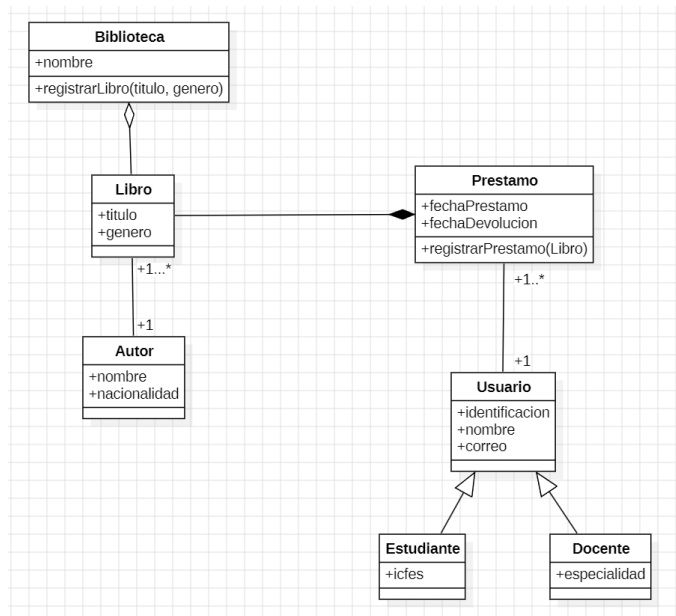
Clases identificadas:

- Biblioteca
- Libro
- Autor
- Usuario
- Estudiante
- Docente
- Préstamo

Relaciones:

- Usuario → Estudiante / Docente (Herencia)
- Biblioteca → Libro (Agregación)
- Libro → Autor (Asociación)
- Usuario → Préstamo (Asociación)
- Préstamo → Libro (Composición)

Diagrama de Clases





Implementación en Python

```
1  class Autor():
    Tabnine | Edit | Test | Explain | Document
2  def __init__(self, nombre, nacionalidad):
3      self.nombre=nombre
4      self.nacionalidad=nacionalidad
5
6  class Libro():
    Tabnine | Edit | Test | Explain | Document
7  def __init__(self, titulo, genero, autor):
8      self.titulo=titulo
9      self.genero=genero
10     self.autor=autor
11
12  class Biblioteca():
    Tabnine | Edit | Test | Explain | Document
13  def __init__(self,nombre):
14      self.nombre = nombre
15      self.libros=[]
16
    Tabnine | Edit | Test | Explain | Document
17  def registrarLibro(self, libro):
18      self.libros.append(libro)
19
```



```
22 class Usuario():
    Tabnine | Edit | Test | Explain | Document
23     def __init__(self, identificacion,nombre,correo):
24         self.identificacion=identificacion
25         self.nombre=nombre
26         self.correo=correo
27
28 class Docente(Usuario):
    Tabnine | Edit | Test | Explain | Document
29     def __init__(self, especialidad,identificacion, nombre, correo):
30         super().__init__(identificacion, nombre, correo)
31         self.especialidad=especialidad
32
33 class Estudiante(Usuario):
    Tabnine | Edit | Test | Explain | Document
34     def __init__(self, icfes,identificacion, nombre, correo):
35         super().__init__(identificacion, nombre, correo)
36         self.icfes=icfes
37
38 class Prestamo():
    Tabnine | Edit | Test | Explain | Document
39     def __init__(self, fechaPrestamo, fechaDevolucion):
40         self.fechaPrestamo=fechaPrestamo
41         self.fechaDevolucion=fechaDevolucion
42         self.prestamos=[]
43
    Tabnine | Edit | Test | Explain | Document
44     def registrarPrestamo(self, titulo, genero, autor):
45         libroPrestamo = Libro("Cien Años de Soledad", "Novela", autor)
46         self.prestamos.append(libroPrestamo)
```



Realizar proceso de Encapsulamiento

Como ejemplo se muestra como se crearían los atributos privados y los métodos para poder acceder y modificar los atributos

```
class Autor():
    Tabnine | Edit | Test | Explain | Document
    def __init__(self, nombre, nacionalidad):
        self.__nombre=nombre
        self.__nacionalidad=nacionalidad

    Tabnine | Edit | Test | Explain | Document
    def obtenerNombre(self):
        return self.__nombre

    Tabnine | Edit | Test | Explain | Document
    def obtenerNacionalidad(self):
        return self.__nacionalidad

    Tabnine | Edit | Test | Explain | Document
    def modificarNombre(self,nombre):
        self.__nombre=nombre

    Tabnine | Edit | Test | Explain | Document
    def modificarNacionalidad(self,nacionalidad):
        self.__nacionalidad=nacionalidad
```

Representación de polimorfismo: El polimorfismo en la Programación Orientada a Objetos (POO) es la capacidad que tienen objetos de distintas clases, derivadas de una superclase común, para responder al mismo método de manera única y específica

```
32 class Usuario():
33     Tabnine | Edit | Test | Explain | Document
34     def __init__(self, identificacion,nombre,correo):
35         self.identificacion=identificacion
36         self.nombre=nombre
37         self.correo=correo
38
39     Tabnine | Edit | Test | Explain | Document
40     def saludar(self):
41         pass
42
43 class Docente(Usuario):
44     Tabnine | Edit | Test | Explain | Document
45     def __init__(self, especialidad,identificacion, nombre, correo):
46         super().__init__(identificacion, nombre, correo)
47         self.especialidad=especialidad
48
49     Tabnine | Edit | Test | Explain | Document
50     def saludar(self):
51         print(f"Desde Docente. Hola soy un objeto de tipo {type(self).__name__}")
52
53 class Estudiante(Usuario):
54     Tabnine | Edit | Test | Explain | Document
55     def __init__(self, icfes,identificacion, nombre, correo):
56         super().__init__(identificacion, nombre, correo)
57         self.icfes=icfes
58
59     Tabnine | Edit | Test | Explain | Document
60     def saludar(self):
61         print(f"Desde Estudiante. Hola soy un objeto de tipo {type(self).__name__}")
```



Actividad Individual

Ejercicio 1 – Análisis y Conversión

Clases:

- Empresa
- Empleado
- Departamento
- Contrato

Relaciones:

- Empresa — Departamento (Agregación)
- Departamento — Empleado (Composición)
- Empleado — Contrato (Asociación)
- Empleado → Administrativo / Operativo (Herencia)

Actividades:

1. Dibujar el diagrama de clases en herramienta seleccionada por ustedes
2. Identificar atributos y métodos.
3. Convertir el modelo a código Python.
4. Aplicar encapsulamiento.
5. Implementar al menos un método polimórfico.

Fecha de Entrega: 04-03-2026

Forma de Entrega: Código fuente de los ejercicios, tanto los trabajados en clase como el solicitado. En repositorio personal. Adicionalmente deben hacer entrega del Diagrama de clases



CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CTPI-CAUCA	03-03-2026